

CAUCASUS UNIVERSITY · SCHOOL OF TECHNOLOGY

Python Programming II

Meeting 3 · Scope & Recursion

nested scope · strings · recursion · induction

Lecturer: **Prof. Maksim Iavich**

miavich@cu.edu.ge · Scientific Cyber Security Association



MEETING 3

Scope & Recursion

Functions that call themselves to solve smaller problems

Nested functions & scope

- A function can be defined inside another function.
- The inner function sees the parameters/variables of the outer one.
- Trace what each call prints — the value depends on which scope a name is looked up in.

Task

Predict the output — default arguments (part 1)

- The same nested function foo is called two different ways.
- Work out by hand what each print statement outputs before running it.

Libraries / focus: *Scope — default parameters & nested functions*

Scope examples — foo / bar (1)

```
def foo(x, y = 5):
    def bar(x):
        return x + 1
    return bar(y * 2)

print(foo(3))
```

y defaults to 5

```
def foo(x, y = 5):
    def bar(x):
        return x + 1
    return bar(y * 2)

print(foo(3, 0))
```

y is passed as 0

Task

Predict the output — shadowing (part 2)

- Now the inner bar has its own parameter x, which shadows the outer x.
- Notice the two versions differ in whether x is passed to bar — trace both.

Libraries / focus: *Scope — variable shadowing*

Scope examples — foo / bar (2)

```
def foo(x):
    def bar(z, x = 0):
        return z + x
    return bar(3, x)

print(foo(2))
```

bar(3, x) — passes x

```
def foo(x):
    def bar(z, x = 0):
        return z + x
    return bar(3)

print(foo(5))
```

bar(3) — x defaults to 0

Task

Strings — useful methods

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Strings — useful methods

Common string operations you will reuse throughout the course.

```
str1.upper()
str1.isupper()
str1.islower()
str2 = str2.capitalize()
str2.swapcase()
str1.index('e')
str2.index('n')
str2.find('n')
str1.count('e')
str1 = str1.replace('e', '*')
```

Multiplication — recursive solution

- **recursive step: reduce the problem to a simpler/smaller version of the same problem.**
  - a*b = a + a + a + … + a (b times) = a + a*(b-1)
- **base case: keep reducing until you reach a simple case that can be solved directly.**
  - when b = 1, a*b = a

Task

Recursive multiplication

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Recursive multiplication

```
def mult(a, b):
    if b == 1:              # base case
        return a
    else:
        return a + mult(a, b-1)   # recursive step
```

Task

Iterative multiplication (for comparison)

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Iterative multiplication (for comparison)

```
def mult_iter(a, b):
    result = 0
    while b > 0:
        result += a
        b -= 1
    return result
```

Factorial

- **n! = n·(n-1)·(n-2)·(n-3)·…·1**
- What n do we know the factorial of? n = 1 → base case: if n == 1: return 1
- How to reduce the problem? Rewrite in terms of something simpler to reach the base case.
- **n·(n-1)! → recursive step: else: return n*factorial(n-1)**

Task

Recursive factorial

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Recursive factorial

```
def fact(n):
    if n == 1:
        return 1
    else:
        return n*fact(n-1)

print(fact(4))
```

Task

Iteration vs. recursion

Write (or trace) the Python program described by the title above.

The worked solution is on the next slide — try it yourself first.

Iteration vs. recursion

```python
def factorial_iter(n):
    prod = 1
    for i in range(1, n+1):
        prod *= i
    return prod
```

Iterative

```python
def factorial(n):
    if n == 1:
        return 1
    else:
        return n*factorial(n-1)
```

Recursive

Task 1

iterPower and recurPower

- Write an iterative function iterPower(base, exp) that computes base**exp using successive multiplication.
- For example, iterPower(base, exp) should compute base**exp by multiplying base times itself exp times.
- Then write a function recurPower(base, exp) which computes base**exp by recursively calling itself on a smaller version of the same problem, and then multiplying the result by base.

Libraries / focus: *Core Python — iteration vs. recursion*

Solution — recurPower

```
def recurPower(base, exp):
    '''
    base: int or float.
    exp: int >= 0
    returns: int or float, base^exp
    '''
    if exp == 0:
        return 1
    return base * recurPower(base, (exp-1))

print(recurPower(5, 3))
```

Mathematical induction

- **To prove a statement indexed on integers is true for all values of n:**
  - Prove it is true when n is the smallest value (e.g. n = 0 or n = 1).
  - Then prove that if it is true for an arbitrary value of n, it must be true for n+1.

Task 2

★ NEW

Recursively reverse a string

- Write a recursive function reverse(s) that returns the string reversed.
- Use it to check whether a password reads the same backwards (a palindrome) — such passwords are weak and predictable.

Libraries / focus: *Recursion + strings — palindrome / weak-password check*

Solution — recursive reverse & palindrome

★ NEW

```
def reverse(s):
    if len(s) <= 1:         # base case
        return s
    return reverse(s[1:]) + s[0]   # move first char to the end

def is_weak_palindrome(pw):
    return pw == reverse(pw)

print(reverse('P@ssw0rd'))
print(is_weak_palindrome('level'))      # True  -> weak
print(is_weak_palindrome('Str0ng!'))    # False
```

Task 3

★ NEW

Recursive hash chain

- A hash chain applies a hash function repeatedly: h, h(h), h(h(h))…
- Write a recursive function hash_chain(text, n) that hashes text n times with SHA-256.
- Hash chains are used in one-time-password schemes and password storage hardening.

Libraries / focus: *Recursion + hashlib — key stretching idea*

Solution — recursive hash chain

★ NEW

```
import hashlib

def sha(text):
    return hashlib.sha256(text.encode('utf-8')).hexdigest()

def hash_chain(text, n):
    if n == 0:            # base case: no more hashing
        return text
    return hash_chain(sha(text), n - 1)   # hash once, recurse

print(hash_chain('secret', 1))
print(hash_chain('secret', 3))   # hashed three times
```



END OF MEETING 3

Recap

Base case + recursive step — the shape of every recursive tool